

COMPTE RENDU

Séance 2 – Création d'images et réseau Docker

Module Conteneurisation – Bachelor

Module	Conteneurisation
Séance	Séance 2 – 4h
Date	21 mai 2026
Groupe	Karadag Nissa • Zemmar Safaa • Mallipoudy Sriram • El Abdallaoui Sami

1. Objectifs de la séance

Cette séance avait pour but de mettre en pratique les concepts suivants :

- Créer une image Docker personnalisée via un Dockerfile
- Déployer une application web conteneurisée avec Nginx
- Configurer un réseau Docker personnalisé
- Mettre en place un volume persistant
- Faire communiquer plusieurs conteneurs entre eux

2. Mise en place de l'environnement

2.1 Arborescence du projet

Le répertoire de travail a été créé et structuré depuis le terminal :

```
C:\Users\karab>cd "C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation"
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation>mkdir tp2-docker
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation>cd tp2-docker

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>mkdir site

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>cd site
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker\site>notepad index.html
```

Création du répertoire tp2-docker et du sous-dossier site/

```

Répertoire de C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker
21/05/2026  11:23    <DIR>        .
20/05/2026  12:26    <DIR>        ..
21/05/2026  11:23         41 .dockerignore

```

Contenu du répertoire tp2-docker (avec le fichier .dockerignore)

```

Fichier  Modifier  Affichage

.git
node_modules
*.log
Dockerfile.old

```

Contenu du fichier .dockerignore

2.2 Fichier index.html

Un fichier HTML simple a été créé pour simuler l'application web :

```

<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>TP Docker</title>
</head>
<body>

  <h1>Mon serveur Docker fonctionne</h1>
  <p>TP 2 conteneurisation</p>

</body>
</html>

```

Contenu du fichier site/index.html

3. Création de l'image Docker

3.1 Dockerfile

Depuis le dossier tp2-docker, le Dockerfile a été créé avec Notepad :

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker\site>cd ..  
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>notepad Dockerfile  
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

Commande de création du Dockerfile

```
FROM nginx:alpine  
  
RUN echo "Configuration Docker OK"  
  
WORKDIR /usr/share/nginx/html  
  
COPY site/index.html .  
  
EXPOSE 80  
  
CMD ["nginx", "-g", "daemon off;"]
```

Contenu du Dockerfile

Le Dockerfile utilise nginx:alpine (image légère ~20 MB) et copie index.html dans le répertoire web. Chaque instruction a un rôle précis :

- FROM nginx:alpine — image de base légère
- WORKDIR — définit le répertoire de travail dans le conteneur
- COPY — copie le fichier HTML local dans le conteneur
- EXPOSE 80 — documente le port écouté par Nginx
- CMD — démarrage de Nginx en mode foreground

3.2 Construction de l'image

Commande utilisée : docker build -t mon-site .

```

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker build -t mon-site .
[+] Building 7.4s (8/8) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 111B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  2.1s
=> [auth] library/nginx:pull token for registry-1.docker.io     0.0s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.2s
=> => transferring context: 288B                                  0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd 3.3s
=> => resolve docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd 0.1s
=> => sha256:94de7c34b8d25a6e2241d9f9af135eba064193cecalb6b782e80d9d8738c24 20.28MB / 20.28MB 1.2s
=> => sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5 1.39kB / 1.39kB 0.2s
=> => sha256:86e0029eccff957b151ddd0fb727628e9ae5fcd85cb9771e74d471efe031245 1.21kB / 1.21kB 0.3s
=> => sha256:bcace0045394f92e9a583eb4f11a674f91cc91d407b85d52afe3f6209672ca1e 400B / 400B 0.3s
=> => sha256:04a7ff7b108a8f7d67888aa7f32283bcda09520660a1d149bd4e6a294cf8b135 951B / 951B 0.1s
=> => sha256:2c084cd9252c2e4b305bd605f0b6534943993f25adfb92dec27d9f2dcca45a2 625B / 625B 0.1s
=> => sha256:16e661a44239bffc66a00e3a0d6c45af7059317a60a5685760937a68d0b3b624 1.88MB / 1.88MB 0.3s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 0.5s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.3s
=> => extracting sha256:16e661a44239bffc66a00e3a0d6c45af7059317a60a5685760937a68d0b3b624 0.3s
=> => extracting sha256:2c084cd9252c2e4b305bd605f0b6534943993f25adfb92dec27d9f2dcca45a2 0.1s
=> => extracting sha256:04a7ff7b108a8f7d67888aa7f32283bcda09520660a1d149bd4e6a294cf8b135 0.1s
=> => extracting sha256:bcace0045394f92e9a583eb4f11a674f91cc91d407b85d52afe3f6209672ca1e 0.1s
=> => extracting sha256:86e0029eccff957b151ddd0fb727628e9ae5fcd85cb9771e74d471efe031245 0.1s
=> => extracting sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5 0.1s
=> => extracting sha256:94de7c34b8d25a6e2241d9f9af135eba064193cecalb6b782e80d9d8738c24e8 0.6s
=> [2/2] COPY site/index.html /usr/share/nginx/html/index.html 0.4s
=> exporting to image                                           0.9s
=> => exporting layers                                           0.5s
=> => exporting manifest sha256:3282edd49341504297a26641c1a39f3e7d632030d3f15734a9e3b913843d3 0.1s
=> => exporting config sha256:ecf8fe9bbca0ad162fc0622c7e3ce63948ab3c290994b44e03cb48af7ad17cd 0.1s
=> => exporting attestation manifest sha256:36d10ca45e37917637c0c937b2701620270200db1c94dfa48 0.1s
=> => exporting manifest list sha256:76312e6676bfa102a14alldf2bf58d9daf224bbb30c0a987da95acdb 0.1s
=> => naming to docker.io/library/mon-site:latest              0.0s
=> => unpacking to docker.io/library/mon-site:latest           0.1s

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>

```

Résultat du docker build — image construite en 7.4s (8 étapes)

4. Réseau Docker personnalisé

Un réseau de type bridge a été créé pour isoler et faire communiquer les conteneurs :

```

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker network create mon-reseau
e131af39163a7433c6b9423a7f8b5b64921aacbb5a8683c89320cba473f4a41d

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>

```

Création du réseau mon-reseau

```

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
8a928b48d682    bridge    bridge      local
0260c974b56d    host      host        local
e131af39163a    mon-reseau bridge      local
51990579827a    none      null        local

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>

```

Vérification via docker network ls — mon-reseau visible avec le driver bridge

5. Volume Docker persistant

Un volume nommé a été créé pour assurer la persistance des fichiers web :

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker volume create mon-volume
mon-volume
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

Création du volume mon-volume

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker volume ls
DRIVER      VOLUME NAME
local       mon-volume
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

Vérification via docker volume ls — le volume mon-volume est présent

6. Lancement du conteneur Nginx

Le conteneur principal a été lancé avec toutes les options requises :

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker run -d --name mon-nginx --network mon-reseau -v mon-volume:/usr/share/nginx/html -p 8080:80 mon-site
65d040d12b528c49bb2985335452a3e389e8446f8a607838dc13306f7d515032
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

Commande docker run avec réseau, volume et mapping de port

Détail des options utilisées :

- -d — exécution en arrière-plan (mode détaché)
- --name mon-nginx — nom explicite du conteneur
- --network mon-reseau — connexion au réseau personnalisé
- -v mon-volume:/usr/share/nginx/html — montage du volume persistant
- -p 8080:80 — redirection du port 8080 de l'hôte vers le port 80 du conteneur

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
65d040d12b52   mon-site   "/docker-entrypoint..." 50 seconds ago Up 49 seconds  0.0.0.0:8080->80/t
cp, [::]:8080->80/tcp   mon-nginx
f083e39ae778   ubuntu    "bash"                  2 hours ago   Up 2 hours
ubuntu-test
```

Vérification via docker ps — conteneur mon-nginx actif, port 8080 exposé

Le site est accessible à l'adresse <http://localhost:8080> :



Page web affichée dans le navigateur — « Mon serveur Docker fonctionne »

7. Communication entre conteneurs

7.1 Création du conteneur Ubuntu

Un second conteneur Ubuntu a été lancé sur le même réseau :

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker run -dit --name ubuntu-test --network mon-reseau ubuntu bash
5e128eff696432058b2b728198bb17f0b2f10b181a1ccd0641ace6f9a8f6b3bb
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

Lancement du conteneur ubuntu-test sur mon-reseau

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
0fe045923688   mon-site   "/docker-entrypoint..." 43 seconds ago Up 42 seconds 0.0.0.0:8080->80/tcp
cp, [::]:8080->80/tcp   mon-nginx
f083e39ae778   ubuntu     "bash"                   2 hours ago   Up 2 hours
ubuntu-test
```

docker ps — les deux conteneurs mon-nginx et ubuntu-test sont actifs

7.2 Test de connectivité

Depuis ubuntu-test, l'outil ping a été installé (apt install iputils-ping) puis utilisé pour joindre mon-nginx par son nom DNS interne :

```
root@5e128eff6964:/# ping mon-nginx
PING mon-nginx (172.18.0.2) 56(84) bytes of data.
64 bytes from mon-nginx.mon-reseau (172.18.0.2): icmp_seq=1 ttl=64 time=0.133 ms
64 bytes from mon-nginx.mon-reseau (172.18.0.2): icmp_seq=2 ttl=64 time=0.241 ms
64 bytes from mon-nginx.mon-reseau (172.18.0.2): icmp_seq=3 ttl=64 time=0.194 ms
^C
--- mon-nginx ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2016ms
rtt min/avg/max/mdev = 0.133/0.189/0.241/0.044 ms
root@5e128eff6964:/#
```

Ping de mon-nginx depuis ubuntu-test — 3 paquets reçus, 0% de perte

Résultat : 3 paquets transmis, 3 reçus, 0% de perte, latence moyenne 0.189 ms. La résolution DNS interne de Docker fonctionne correctement.

8. Vérifications et logs

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker logs mon-nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/20 09:48:05 [notice] 1#1: using the "epoll" event method
2026/05/20 09:48:05 [notice] 1#1: nginx/1.31.0
2026/05/20 09:48:05 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/05/20 09:48:05 [notice] 1#1: OS: Linux 6.6.114.1-microsoft-standard-WSL2
2026/05/20 09:48:05 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/20 09:48:05 [notice] 1#1: start worker processes
2026/05/20 09:48:05 [notice] 1#1: start worker process 30
2026/05/20 09:48:05 [notice] 1#1: start worker process 31
2026/05/20 09:48:05 [notice] 1#1: start worker process 32
2026/05/20 09:48:05 [notice] 1#1: start worker process 33
2026/05/20 09:48:05 [notice] 1#1: start worker process 34
2026/05/20 09:48:05 [notice] 1#1: start worker process 35
2026/05/20 09:48:05 [notice] 1#1: start worker process 36
2026/05/20 09:48:05 [notice] 1#1: start worker process 37
2026/05/20 09:48:05 [notice] 1#1: start worker process 38
2026/05/20 09:48:05 [notice] 1#1: start worker process 39
2026/05/20 09:48:05 [notice] 1#1: start worker process 40
2026/05/20 09:48:05 [notice] 1#1: start worker process 41
172.18.0.1 - - [20/May/2026:09:48:14 +0000] "GET / HTTP/1.1" 200 216 "-" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"

What's next:
View and search logs for all containers in one place
with Docker Desktop's Logs view. docker-desktop:///dashboard/logs

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

Logs du conteneur mon-nginx (docker logs mon-nginx) — démarrage Nginx et requête HTTP reçue

Autres vérifications effectuées :

- docker ps — liste les conteneurs actifs
- docker network ls — confirme la présence du réseau mon-reseau
- docker volume ls — confirme la présence du volume mon-volume
- docker images — mon-site:latest visible (92.7 MB)

9. Architecture finale

```
[ Navigateur ] → localhost:8080
      ↓
[ mon-nginx ]  (image: mon-site, port 80)
      ↑
      mon-reseau (bridge)
[ ubuntu-test ] (ping mon-nginx → OK)
```

10. Conclusion

Cette séance a permis de mettre en pratique l'ensemble du cycle de vie d'une image Docker : création via Dockerfile, construction, déploiement avec réseau et volume, et communication inter-conteneurs.

Les quatre objectifs de la séance ont été atteints :

- Image personnalisée mon-site construite à partir de nginx:alpine
- Réseau Docker mon-reseau opérationnel avec résolution DNS interne
- Volume mon-volume monté pour la persistance des données
- Communication validée entre mon-nginx et ubuntu-test via ping